

Problem C:

The FloorSet Challenge: Data-Driven SoC

Floorplanning

*Uday Mallappa, Sachin Bhat, Pei Chun Ch'ng, Somdeb Majumdar,
Yuan Lu, Yusu Wang
(Intel)*

**To facilitate centralized issue tracking and faster response times,
you are encouraged to submit your questions directly through the Git repository.**

Q&A

Q1. The write-up mentions that any deviation in dimensions for fixed-shape and preplaced blocks is a hard-violation and should result in a score of 10, but then there is also a soft-constraint formula given to compute these violations, which are contradictory in nature. Could you please specify whether to consider this as a hard or soft constraint?

cost.py provided in the GitHub repo doesn't explicitly consider hard-constraint penalty ($M=10$) or the alpha, beta and gamma values given in the cost function. Does this mean that the organizers are yet to provide a proper validator program?

A1. Here is the contest repo, that addresses these concerns.

<https://github.com/IntelLabs/FloorSet/tree/main/iccad2026contest>

Q2. I am writing to request clarification regarding the technical environment and evaluation process for Problem C.

Specifically, I would like to inquire about the following details:

1. Memory Capacity: Could you please provide the specific memory (RAM/VRAM) limits allocated for each submission during evaluation?
2. Batch Processing: During the validation and test phases, can the test cases be processed in batches, or they should be evaluated sequentially?

A2. These are the hardware specs -> A100 80GB VRAM GPU, ICELAKE CPU with 48 cores, 128GB RAM

Batch processing -> The test cases will be evaluated sequentially.

Q3. I am writing to clarify the hardware specifications for the hidden test set evaluation environment, as this heavily impacts our inference strategy and the "Runtime Factor" score.

Could you please provide the following details regarding the evaluation server:

1. What specific CPU and GPU models will be used for evaluation?
2. What is the maximum RAM capacity allocated per submission?
3. Are we permitted to utilize multiprocessing or multithreading during inference?

A3.

1. A100 80GB VRAM GPU, ICELAKE CPU with 48 cores
2. 128GB RAM
3. The test samples are evaluated sequentially. But for each sample evaluation, multiprocessing or multithreading can be used.

Q4. We found that the training data golden answers violate MIB, Boundary, and Group constraints in some cases.

Since these are soft constraints, are the golden answers themselves penalized?

A4. Yes, seeing soft violations in training data is normal and consistent with how the dataset is intended to be used.

Q5. I am writing to ask about how to handle a potential constraint conflict in some test cases.

In particular, I found that in some examples, two modules may satisfy the following conditions at the same time:

They belong to the same boundary constraint.

They are both preplaced modules.

One of the preplaced modules does not touch the corresponding boundary.

In this case, if we strictly enforce both the preplaced-position constraint and the

boundary constraint, the problem appears to become infeasible. Could you please clarify how such cases are expected to be handled during evaluation?

Should we:

Prioritize keeping the preplaced modules fixed and allow the boundary constraint to be partially violated;

Prioritize satisfying the boundary constraint and adjust the preplaced module accordingly;

Treat the case as infeasible;

Or follow some other specified rule?

I would appreciate your clarification on the expected handling of such conflicting constraints so that we can make our implementation consistent with the evaluation rules.

A5. In the contest evaluator, preplaced position is a hard constraint, and boundary touching is a soft constraint. They operate at different levels and there is no ambiguity about priority.

Preplaced position (hard constraint): A preplaced block must appear at exactly the (x, y, w, h) specified in the input. Any deviation makes the solution infeasible and assigns $\text{Cost} = M = 10$ for that test case. This cannot be overridden by any soft constraint.

Boundary touching (soft constraint): Whether a block touches its required boundary edge or corner is checked as part of V_{soft} . A violation increments the boundary violation count, which raises the exponential penalty term $\exp(\beta \cdot V_{\text{relative}})$. The solution remains feasible.

The correct handling: Keep all preplaced modules at their fixed positions. If a preplaced module's fixed position does not touch its boundary constraint, that boundary violation is counted in V_{soft} and penalized through the cost function, but the solution is still feasible. You should never move a preplaced module to satisfy a soft constraint.

Q6. For the soft blocks, can they exceed their area target? If the area target is 100, can the block be 12x12? The reason I ask is that in order to get abutment between a pair of blocks, or to have a block touch the side or top of the placement area, it might be useful to stretch the block in one or more directions.

The contest specifies a 1% area tolerance, and going below the target area is clearly a violation, but it seems reasonable to allow things to be larger.

A6. No. The 1% area tolerance is enforced symmetrically in both directions using an absolute relative error check:

$$|w \times h - \text{area_target}| / \text{area_target} \leq 0.01$$

Both exceeding and falling short of the target by more than 1% count as a hard constraint violation, making the solution infeasible (Cost = M = 10). A block with target area 100 placed as 12×12 (area = 144, +44% excess) would be infeasible.

The rationale is that soft blocks have a target area reflecting the actual silicon area required by the module. Allowing unconstrained growth would let a solver trivially satisfy grouping, MIB, and boundary soft constraints by inflating block sizes, circumventing the intent of those constraints. The 1% tolerance exists only to accommodate minor aspect-ratio adjustments while preserving the overall die area budget.

If you need a block to touch a boundary or abut a neighbor, the correct approach is to adjust its aspect ratio (width/height) while keeping $w \times h$ within 1% of the target area, not to increase the area itself.

Q7. The submission guidelines state that participants should submit an optimizer executable (with PyInstaller recommended for Python submissions), while the provided evaluation script appears to import an optimizer.py file, instantiate the MyOptimizer class, and invoke its solve() method directly.

This raises some uncertainty about how executable-based submissions will be evaluated. Specifically, if participants submit a standalone binary executable instead of source code, it is unclear how the evaluation framework will interface with the optimizer, since the currently available evaluation script expects a Python module and class interface.

Could you please clarify:

1. Whether the final evaluation will still use the same Python evaluation script and import-based interface (optimizer.py with a MyOptimizer class and solve() method).
2. If executable submissions are supported, how the evaluation system will invoke the executable and exchange inputs/outputs with it.

3. Whether participants should prioritize compatibility with the provided evaluation script or package their solution solely as an executable.

The guidelines mention that evaluation will be performed "in the same command format provided in the evaluation script," which seems to suggest the Python interface will be used, but we would like to confirm this interpretation.

A7. We will wrap the participants' executable with a python script during evaluation. The focus for participants must be toward packaging their solution as an executable with all the dependencies.

Q8. How should the submitted executable receive input data (e.g., command-line arguments, stdin, or input files)?

A8. Adhere to same input schema of the solve() method in optimizer_template.py

Q9. How should the executable return the floorplanning results (e.g., stdout or output files), and what is the expected format?

A9. Adhere to same output schema of the solve() method in optimizer_template.py

Q10. Will the final evaluation still require compatibility with the MyOptimizer.solve() interface?

A10. Yes

Q11. Could you provide an executable interface specification or an example wrapper script so that participants can properly test their packaged solutions before submission?

A11. An example (op_wrapper.py) is attached. This wrapper will invoke the executable. Evaluation of hidden test cases using this command: python iccad2026_evaluate.py --evaluate op_wrapper.py

Q12. Regarding Problem C, I would like to ask whether the checker used in the Beta Test and Final Test will be the same as the one used in the Alpha Test.

A12. Beta test and final test will use the hidden testcases. Alpha testing was pipecleaning and we did not want the candidates to get confused over the score computation, and it was primarily meant for reproducing the experiment on our end.

Q13. If applying $\max(0, \text{gap})$ before plugging them into the cost formula, we analyzed the alpha test results and noticed a runtime discrepancy:

The top 5 teams have solution runtimes exceeding 200s or even 500s, while the median runtime across all alpha test submissions is at most 11 seconds.

Based on the given cost formula, such extreme runtime factors will heavily inflate the overall Cost value, making it nearly impossible for these slow top-performing teams to reach a Cost score of 0.8–0.9.

We would like to confirm:

Is the runtime factor term not activated or weighted during alpha test scoring?

A13. The provided leaderboard is the runtime for all 100 testcases. The median runtime file is per-testcase.

Q14. We use PyInstaller to pack up our optimizer. But we found that for the start of the evaluation for every design, the time to import torch and load checkpoints takes a long time. We want to ask if these cold start are counted in the final running time or just the algorithm itself. Thanks !

A14. No, we do not use library and checkpoint loading times for runtime computation.

Q15. We would like to ask for further clarification regarding the intended balance between runtime and physical-design quality.

We currently have two solution variants:

Variant A:

- Cost before runtime adjustment: 2.900978
- Average runtime: 1.052 seconds
- Estimated score after applying the published per-testcase median runtimes: 2.0434

Variant B:

- Cost before runtime adjustment: 1.911589
- Average runtime: 8.239 seconds
- Estimated score after applying the published per-testcase median runtimes: 2.4701

Variant B takes approximately 7.19 additional seconds on average and is approximately 7.8 times slower. However, its cost before runtime adjustment is approximately 34% lower, indicating a substantial overall improvement in HPWL, bounding-box area, soft-constraint satisfaction, or a combination of these quality-related metrics.

Nevertheless, after applying RuntimeFactor, Variant A receives the better contest score. In this case, an average difference of approximately seven seconds is sufficient to reverse the ranking despite the significant quality-related improvement achieved by Variant B.

We understand that the official RuntimeFactor is calculated independently for each testcase as follows:

RuntimeFactor for testcase i = submission runtime on testcase i / median runtime of all submissions on testcase i .

Therefore, the average runtimes above are provided only to illustrate the overall difference between the two approaches. Our estimated final scores were calculated using the currently published per-testcase median runtimes.

Our main question is whether this degree of ranking reversal is consistent with the intended design of the scoring system.

We are particularly interested in the effect this may have among the top-ranked submissions. When submissions reach the top of the leaderboard, their HPWL, area, and soft-constraint results may already differ by only a few percentage points.

Although the runtime exponent is sublinear, the minimum runtime multiplier of 0.7 still provides up to a 30% reduction in the complete cost. This may therefore become one of the dominant factors determining the final ranking.

For example, consider two feasible submissions:

- Submission X has a quality-only cost of 1.30 and receives the minimum runtime multiplier of 0.7.
- Submission Y has a quality-only cost of 1.00 and runs at the median runtime, giving it a runtime multiplier of 1.0.

After applying the runtime multiplier:

Submission X final cost = $1.30 \times 0.7 = 0.91$.

Submission Y final cost = $1.00 \times 1.0 = 1.00$.

Submission X would therefore rank above Submission Y even though its quality-only cost is 30% worse.

A related concern is that both the HPWL gap and the area gap are clamped from below at zero. As a result, a solution cannot receive additional quality credit for outperforming the official baselines.

For example, suppose a submission achieves the following:

- HPWL gap: -10%
- Area gap: -5%
- Soft-constraint violations: 0

Although this solution outperforms the official baseline in both HPWL and area, both negative gaps are treated as zero in the scoring formula. Its quality-only cost therefore remains 1.0, exactly the same as that of a solution that merely matches both baselines. If this above-baseline solution runs at the median runtime, its final cost remains:

$$\text{Final cost} = 1.00 \times 1.00 = 1.00.$$

In contrast, another solution with a quality-only cost of 1.40 could obtain:

$$\text{Final cost} = 1.40 \times 0.70 = 0.98.$$

Therefore, a faster solution with noticeably worse physical-design quality could rank above a solution that not only reaches the theoretical minimum quality score, but actually outperforms the official HPWL and area baselines.

More generally, because $1.00 / 0.70$ is approximately 1.4286, a fast solution with a quality-only cost of up to approximately 1.4286 could theoretically outrank a perfect-quality solution running at the median runtime.

We understand that the 0.7 lower bound is intended to limit the maximum benefit from speed and prevent unlimited incentives for further runtime reduction. However, because the runtime multiplier is applied directly to the complete quality and violation cost, the maximum 30% discount can still outweigh a substantial difference in physical-design quality.

We would also like to ask about the possible effect on hidden-test evaluation.

The currently published per-testcase median runtimes are based on the public testcases. Public data naturally serves as an important basis for algorithm development, tuning, and analysis. As a result, some approaches may become more heavily optimized around the characteristics and runtime behavior of the public cases. Other approaches may spend additional computation on broader search, legality verification, correction, and refinement in order to improve robustness and generalization to unseen instances.

On hidden testcases, particularly those that are more difficult or have somewhat different characteristics, a robust method may require additional computation to maintain solution quality. RuntimeFactor could then penalize the method partly because it spends more time carefully processing these difficult cases.

Meanwhile, a method that places stronger emphasis on minimizing runtime may receive a favorable runtime multiplier even if its solution quality generalizes less effectively to unseen testcases.

We fully agree that efficiency and generalization are not mutually exclusive. Ideally, a strong method should achieve high solution quality, robustness, and efficiency simultaneously. We also agree that genuine algorithmic improvements, such as better data structures, incremental evaluation, parallelism, and effective hardware/software utilization, should be rewarded when they reduce runtime without sacrificing quality. Our concern is mainly whether the current formula may encourage participants, when selecting their final submission, to choose a noticeably lower-quality result in order to obtain a larger runtime discount, even when the higher-quality method requires only several additional seconds.

From an industrial perspective, optimization runtime is generally a one-time or limited design-stage cost. In contrast, differences in chip area, wirelength, power, timing, congestion, yield, or other physical-design metrics may affect the entire downstream design flow and may be reflected repeatedly across every manufactured chip.

In practical design scenarios, it may therefore be reasonable to spend several additional seconds, minutes, or even longer during optimization in exchange for better final PPA and a more robust physical-design result.

We understand that a contest scoring objective does not need to exactly reproduce an industrial business-cost model. Nevertheless, we would appreciate clarification on whether the resulting trade-off reflects the research direction that the contest intends to promote.

Specifically, we would like to ask:

1. Is RuntimeFactor intentionally allowed to reverse the ranking of two submissions when their quality-only costs differ substantially?
2. Is the minimum runtime multiplier of 0.7 intended to allow a faster solution to outrank a solution that has already achieved the theoretical minimum quality score?
3. Since negative HPWL and area gaps are clamped to zero, should runtime be able to outweigh improvements that exceed the official physical-design baselines but are not otherwise reflected in the quality score?
4. For top-ranked submissions whose physical-design quality is already very close, is RuntimeFactor intended to become one of the main factors determining their ranking?
5. In the hidden-test evaluation, have the organizers considered that differences in testcase difficulty or distribution may increase the runtime of methods that perform more extensive search and validation to improve robustness and generalization?

6. Would the organizers consider publishing a quality-only cost without runtime adjustment, or separate reference rankings for solution quality and runtime efficiency, while retaining the current combined score as the official ranking?

We fully understand and respect the published scoring rules, and we will continue optimizing our submission according to the official formula. Our intention is not to disregard computational efficiency, but to better understand whether the practical ranking behavior caused by RuntimeFactor, particularly the minimum multiplier of 0.7, the clamping of negative quality gaps, and the possible behavior on hidden testcases, matches the contest's intended balance among physical-design quality, generalization, and algorithmic efficiency.

A15. Your reading of the published formula is correct: runtime effect is $\max(0.7, \text{RuntimeFactor}^{0.3})$, HPWL and area gaps are clamped at zero from below (so beating the baseline does not further reduce cost), and among submissions that already match baseline quality with low soft violations, relative speed can materially affect ranking. That behavior follows from the specification document.

A few points help put the concern in context. Examples provided to one testcase may not reflect the full picture. The official score aggregates all 100 testcases (with higher weight on larger-partition testcases). Also, the final ranking is based on the hidden test set that will be out during Beta phase.

The contest is intended to encourage both strong results (vs baseline and constraints) and computational efficiency relative to other participants. We recognize that this tradeoff differs from some industrial settings where optimization time is less critical than optimization objectives.

There will be no change to the official scoring formula for this contest. We hope the beta evaluation on the hidden testcases gives you a clearer view, and that it helps you decide which submission to prioritize going forward.

Q16. When we validate the training dataset, we found an inconsistency between `tree_sol` and `fp_sol`:

If we adopt the block width/height values from `fp_sol`, we cannot reconstruct the matching layout of the `fp_sol` from the corresponding `tree_sol`. This may indicate potential errors in the `tree_sol` data records.

We have two follow-up questions here:

Is the `tree_sol` in the released training dataset confirmed to contain incorrect values?

If confirmed defective, will you release a revised, fixed version of the training dataset with corrected `tree_sol` before the final submission deadline?

A16. For supervised learning or imitation, treat `fp_sol` as the ground-truth target layout: per-block size and position (`n_blocks × 4`, stored as width, height, x, y in the Lite loader). The contest evaluator and the provided training example use `fp_sol` (and metrics for baselines during training), not `tree_sol`.

`tree_sol` is a supplementary **datapoint, in the form of B*-tree**. It is valid **for the same floorplan as the paired `fp_sol`** when you decode it with the **same tree semantics and insertion/traversal order** used when the dataset was built. If you decode a tree with a different convention, the reconstructed layout will not match `fp_sol`. If you use decide to use `tree_sol`, follow the **tree → layout** conversion from [IntelLabs/parsac](https://github.com/IntelLabs/parsac). Again, you do not need `tree_sol` for the contest submission.